Emotion-Based Music Player

Using ChatGPT with Conversational Mood detection.

Submitted by: Ruairí McClintock
Student number: R00203119
Date: 28/04/2025

Supervisor: Martin Klepal
Assessor: Fergus Reilly

Department of Electrical &Electronic Engineering

Declaration

I declare that this submission is my own work and writing.

Any text, diagrams or other material copied from other sources (including, but not limited to, books, journals and the internet) have been clearly acknowledged and referenced as such in the text by the use of 'quotation marks' (or indented *italics* for longer quotations) followed by the author's name and date [e.g. (Byrne, 2008)] either in the text or in a footnote/endnote. These details are then confirmed by a fuller reference in the bibliography section.

I acknowledge that I understand the concept of plagiarism. I have studied MTU's guide to referencing (Section 3 Referencing at http://www.mycit.ie/libraryguide/) and I have read the section on Plagiarism and Cheating (pages 46 – 49) in MTU's Student Regulations handbook (available at http://www.mycit.ie/). I understand that only assignments which are free of plagiarism will be awarded marks. By submitting this assignment I agree to these terms. I further understand that CIT has a plagiarism policy which can lead to the suspension or permanent expulsion of students in serious cases.

I understand that my project report will be evaluated by the Turnitin tool in Canvas to detect instances of plagiarism.

Signed:

Date: 23/05/2025

Table of Contents

Declaration i

Table of Contents ii

List of Figures v

List of Tables vi

Abstract vii

Acknowledgements (Optional) viii

1. Introduction 1

1.1: Project Overview 1

1.2: Importance of the Problem 1

1.4: The Need for a New Solution 2

1.5: Aims and Objectives 2

2. Literature Review/Theoretical Background 3

2.1: Prompt Engineering for Large Language Models in Emotion Recognition . 3

2.2: Social Tagging Systems (Last.fm) for Music Mood Classification 4

2.3: Online (Cloud) Music Players and Modular Architecture in MIR 5

2.4: Alternative Approaches and Critical Perspective 6

2.6: Conclusion. 7

3. System Design 7

3.1: Solution Overview 7

3.2: High-Level Architecture 8

3.3: ChatGPT (Chat_Bot) Emotion detection module. 8

3.3.1 Prompt Engineering 9

3.3.2 Code implementation. 11

3.4: Last.fm (Mood_music) Music recommendation module. 12

3.4.1: Why Last.fm 13

3.4.2 Tag to Track Mapping logic. 13

3.4.3: Code Implementation 14

3.5: Spotify (Music_player) Module. 15

3.5.1 Playback Flow and Track Matching Logic 15

3.5.2 Code Implementation 16

3.7: GUI Module. 16

3.7.1: GUI Interaction flow. 18

3.7.2: Code Implementation 18

4. Implementation/Simulation/Test/ Realisation 19

4.1: Full-System Testing. 19

4.2: Implementation Revisions. 20

4.3: User flow and Behavioural Testing. 20

4.4 Execution Logging and Validation Strategy. 21

5. Results 25

5.1 Emotion Detection Output 25

5.2 Testing Multi-Emotion Inputs Against Prompt Logic 27

5.3: Error Handling Outputs 29

6. Analysis. 30

6.1 Conversational Emotion Detection 31

6.2 Tag-Based Song Recommendation 31

6.3 System Stability and Error Management 31

6.4 Prompt Effectiveness and Limits 32

6.5 Summary Table of Observed Performance 33

7. Conclusion 34

References 35

List of Figures

Figure 1-High-Level System Architecture Diagram 8

Figure 2-ChatGPT Chat_Bot Block: User emotion input processed into structured JSON tags for music selection. 9

Figure 3-Mood-Music Block: Last.fm receives weighted mood tags and returns a list of 10 recommended songs. 12

Figure 4-Spotify Block Diagram: Receives song_list.json and queues songs to user's active Spotify session. 15

Figure 5-GUI Layout: Left panel for conversation and controls, right panel for logs and outputs. 17

Figure 6-GUI Log Panel Showing JSON Detection and File Confirmation 22

Figure 7-Error message shown to User (left side) 23

Figure 8-Error message shown for debugging (on right side) 23

Figure 9-song_list.json Displayed After Track Generation 24

Figure 10-Confirmation of Track Queueing to Spotify 24

Figure 11-assistant_output.json for 'Tired but Hopeful' Input 26

Figure 12-song_list.json returned from Last.fm using mood tags 26

List of Tables

Table 1-Sudo Interaction between Chatbot and User 25

Table 2-Prompt Step Completion vs Input Complexity 28

Table 3-Testing Summary..... 30

Table 4-Analysis Summary 33

Abstract

This project presents the development of an emotion-based music player that uses ChatGPT for conversational mood detection and integrates Last.fm for mood-tagged music recommendation. The system allows users to describe how they feel in natural language, extracts their emotional state through a structured prompt, and maps it to weighted mood tags. These tags are used to generate a song list, which is then queued for playback using the Spotify API.

A modular design was implemented to separate the chatbot, music selection, and playback functions, ensuring flexibility and ease of maintenance. A custom GUI was developed to manage user input and system feedback, with integrated logging for validation and debugging.

Testing confirmed that the system successfully handled a range of emotional inputs, generated valid JSON outputs, and returned accurate music matches. Errors such as invalid input, missing files, or Spotify issues were handled gracefully. Limitations included the system's inability to resolve vague or off-topic responses and a lack of long-term user adaptation.

Overall, the project demonstrates that conversational AI can be used to build a real-time, emotion-driven music recommendation system using current APIs and lightweight infrastructure.

Acknowledgements (Optional)

I would like to express my sincere thanks and great gratitude to Martin Kelpal who not only helped me in every step of this project but for our brainstorming sessions, his great encouragement, his love for the topic and general discussions around my project. This project would have been a lot harder if not for him.

Ruairí McClintock

May 2025

1. Introduction

1.1: Project Overview

This project focuses on developing an emotion-based music player that uses ChatGPT for mood detection and music recommendations. The system leverages ChatGPT's conversational abilities to detect the user's emotional state through text input. It then uses this emotional data to recommend music, integrating the Last.fm API for music classification based on mood tags and metadata. Music playback is managed via online music APIs like Spotify or Apple Music, ensuring real-time access to a broad catalogue of songs.

1.2: Importance of the Problem

Personalization in music recommendation has become an essential feature of many modern music platforms. Current music recommendation systems, such as those offered by Spotify or Apple Music, primarily rely on user listening history, playlists, and genre-based categories. While these systems are effective at providing general recommendations, they often fail to adapt to the user's dynamic emotional state in real-time.

Music has a powerful impact on emotions and can play a significant role in enhancing mental well-being. However, traditional music recommendation systems are often static, offering mood-based playlists that are too general to reflect the nuanced emotional states a user might experience. The potential for improving these systems lies in tailoring music recommendations to the user's **real-time emotional input**, offering personalized playlists that better align with the user's current mood and emotional needs.

The ability to match music recommendations with real-time emotional states could not only improve the user experience but also have significant mental health benefits. For example, music tailored to an individual's emotional state could help alleviate stress, improve mood, or provide emotional support during difficult times. Despite the growing importance of personalized music experiences, existing systems are limited in this aspect.

1.4: The Need for a New Solution

Existing music recommendation systems primarily rely on historical user data, which fails to address the dynamic and nuanced nature of emotions. The need for a solution that integrates **real-time emotion detection** and **adaptive music recommendations** is critical. By using **AI-powered conversational mood detection** (via ChatGPT), the system can detect a user's emotional state and adjust its recommendations accordingly. This approach allows for a **dynamic** and **personalized** music experience that adjusts to the user's changing emotions in real-time.

Moreover, using **community-driven music tagging systems**, such as Last.fm, allows the system to map **user-perceived emotional states** to specific songs, creating a more accurate and effective mood-matching system. While emotion-based playlists exist, they often fall short in terms of personalization. This project aims to bridge that gap by creating a system that continuously learns and adapts based on **user interaction** and **real-time emotional input**.

1.5: Aims and Objectives

The aim of this project is to develop a working prototype of an emotion-based music player that integrates ChatGPT for mood detection and Last.fm for music recommendations. The specific objectives of the project are:

1. To develop a conversational AI system using ChatGPT that can detect emotions from user input and generate corresponding music recommendations.
2. To integrate the Last.fm API to provide music recommendations based on detected emotions, mapping mood tags to songs.
3. To create a modular system that allows for easy integration with different music APIs, ensuring flexibility in platform choice (e.g., Spotify, Apple Music).
4. To design a user-friendly interface that allows users to input their emotional state and receive personalized music recommendations in real-time.

2. Literature Review/Theoretical Background

2.1: Prompt Engineering for Large Language Models in Emotion Recognition

Prompt engineering is crucial for guiding Large Language Models (LLMs) like ChatGPT to generate structured and accurate outputs, which is particularly important in tasks such as emotion recognition. Theoretical work in this area has shown that concise, well-structured prompts help reduce ambiguity and improve the model's accuracy by focusing the model on specific outputs, such as detecting emotional states from user input and generating corresponding music recommendations [1][2].

Research demonstrates that shorter, more focused prompts improve performance by minimizing the complexity of responses, thus enabling faster and more reliable emotion detection. This approach is aligned with the broader theory of task-specific prompt design, where clarity and precision are prioritized over verbosity [3].

However, edge cases and ambiguous emotions still pose challenges, as even well-structured prompts may not fully capture nuanced user sentiments. Despite these challenges, refined prompts can greatly enhance LLM performance, especially when paired with iterative feedback loops that adjust the model’s understanding over time.

In the context of this project, concise prompt design was chosen as the theoretical approach because it aligns with the literature advocating for efficient, structured output in emotion-driven tasks. This approach minimizes computational load and improves the usability of the system for real-time music recommendations.

2.2: Social Tagging Systems (Last.fm) for Music Mood Classification

The use of Last.fm as a music mood classification engine is grounded in the principles of crowd-sourced tagging systems. Last.fm’s user-generated tags provide a rich source of mood-related metadata, which has been validated through extensive research and is widely regarded in the field of Music Information Retrieval (MIR). Research has shown that the community-driven nature of Last.fm allows for a large and diverse dataset, where mood tags like "happy," "relaxed," and "sad" are applied to songs based on collective user input, creating a scalable and polarized set of mood data. These tags are highly relevant for the task of emotion-based music classification as they are mapped to well-established psychological models of emotion, such as Russell’s circumplex model of affect, which organizes emotions based on two dimensions: valence (positive/negative) and arousal (high/low) [4].

Last.fm's crowd-sourced tagging system offers distinct advantages over other music services, particularly when it comes to mood classification. Unlike Spotify, which primarily focuses on audio features, Last.fm enables the use of user-generated tags that provide valuable insights into the emotional perception of songs. The advantage of these textual, mood-related tags is that they are easier to interpret and are more aligned with psychological models of emotion than purely audio-based features. Moreover, Last.fm's data is continuously updated as users interact with the platform, ensuring that the mood tags remain relevant and adaptive to current trends in music listening.

However, Spotify’s API—historically one of the leading services for music classification—has faced recent changes that limit access to critical data, including audio feature analysis and mood-based recommendations. In November 2024, Spotify announced changes to its Web API, which included the removal of certain endpoints that were previously available to developers. These changes directly impacted access to audio features and mood-based data, which were crucial for creating emotion-based music recommendations. The API changes limited access to detailed song

metadata and required developers to shift their approach toward more generalized features or external datasets [5].

These restrictions, combined with Spotify’s reliance on audio features—which are valuable but less explicit for mood classification—highlight Last.fm’s superiority for mood-based tasks. Unlike Spotify, which focuses on technical attributes like tempo, valence, and energy, Last.fm's tags directly align with user-perceived emotional states, offering a richer, more humanized way to classify music emotions. Moreover, Spotify’s recent API changes have further underscored the importance of community-driven and openly accessible datasets like Last.fm, which remain unaffected by the shifting landscape of music platform APIs.

Therefore, Last.fm was chosen for this project, not only because of its robust mood classification system but also due to the stability and openness of its API compared to Spotify’s increasingly restricted access. While Spotify's service excels in delivering detailed audio analysis, Last.fm’s community-based mood tagging system provides a more precise, scalable, and flexible approach to emotion-based music recommendation, particularly in a system that aims to understand and classify user emotions.

2.3: Online (Cloud) Music Players and Modular Architecture in MIR

In the context of Music Information Retrieval (MIR), modular architecture is a highly recommended design pattern for building scalable, maintainable, and adaptable systems. Modular design ensures that each component of the system (such as emotion detection, music recommendation, playback, and API interactions) is independent and replaceable. This modularity is particularly beneficial in music recommendation systems, where the underlying APIs (such as Spotify or Apple Music) may evolve or change over time. By decoupling the system into distinct modules, it becomes possible to swap out one music service for another without overhauling the entire system.

For example, if Spotify’s API were to become less accessible or less suitable for the system’s needs, it could easily be replaced with an alternative such

as the Apple Music API, which provides a similar set of functionalities for music data retrieval and playback control [6]. Both APIs offer robust support for real-time data and music recommendations, but their integration would not require reworking the entire system.

The flexibility of modular architecture is that it does not require the use of just one API like Apple Music or Spotify—multiple APIs can be integrated simultaneously depending on the user's preferences or needs. For example, a user may prefer Spotify for its music catalogue and features, while another user may prefer Apple Music for its exclusive content. By utilizing a modular design, these preferences can be easily accommodated without adding complexity to the core system. This allows the system to be adaptive, scalable, and future-proof, as it can integrate new music services as they emerge, or replace services if access becomes restricted.

Modular architecture not only simplifies the development and maintenance of the system but also ensures that the system can adapt to changing requirements or service failures with minimal disruption, while still benefiting from the latest technologies and music catalogues.

2.4: Alternative Approaches and Critical Perspective

While the chosen approach of tag-based mood classification supported by LLM-driven emotion detection is widely supported, there are alternative methods that are also effective for emotion-based music recommendation. These include audio feature-based systems, which rely on inherent musical properties (e.g., rhythm, melody) to classify emotions, and hybrid systems that combine audio features and lyrics to provide a more comprehensive understanding of mood [3].

Despite the potential of these alternative methods, tag-based mood classification using LLMs offers the most practical solution for this project. This method allows for scalable, flexible, and real-time music recommendations, while also benefiting from community-driven tagging systems that are already widely used and academically validated.

2.6: Conclusion.

The theoretical foundations laid out above prompt engineering, crowd-sourced tagging systems, and modular online-based architecture collectively form a strong basis for building an emotion-based music recommendation system. These principles ensure the system is scalable, adaptable, and capable of providing real-time, personalized music recommendations based on user emotions. The subsequent sections of the report will demonstrate how these theoretical concepts were applied in practice to create an operational system.

3. System Design

3.1: Solution Overview

The system is designed to interact with the user, detect their emotional state, convert these emotions into mood-specific tags, and then generate personalized music recommendations based on those emotions. The process flows through several stages to provide a seamless music experience:

1. **User Interaction and Emotion Detection (ChatGPT)**:
   The system begins by conversing with the user through ChatGPT. The user inputs their emotional state (e.g., "I feel happy" or "I'm a bit sad"). ChatGPT interprets the user's feelings and asks follow-up questions to refine the emotions.
2. **Emotion to Mood Tags (ChatGPT to Last.fm)**:
   Once the emotions are identified, ChatGPT converts them into Last.fm mood tags, each with an associated weight to reflect the intensity of the emotion. These 10 tags are then saved into a JSON file. This step is essential for mapping the detected emotions to music characteristics.
3. **Music Recommendation (Last.fm)**:
   The JSON file containing the 10 emotion tags is sent to Last.fm. Last.fm, using its crowd-sourced tagging system, maps these tags to mood-based songs. Last.fm returns a list of 10 songs that match the detected mood and emotional tags.

4. **Music Playback (Spotify)**:
   The song list, complete with artist names and titles, is saved to another JSON file, which is then sent to Spotify via its API. The songs are queued in the user's Spotify account and ready for playback.
5. **Real-Time Adaptation (User Feedback)**:
   The system includes a feedback loop where users can confirm whether the music recommendation matches their emotional state. The feedback is used to improve the system's understanding of the user's preferences and fine-tune future recommendations.

3.2: High-Level Architecture

The system architecture is designed to integrate emotion detection, music recommendation, and playback into a cohesive and adaptive user experience. Below is the high-level architecture diagram that illustrates the interaction between the components:

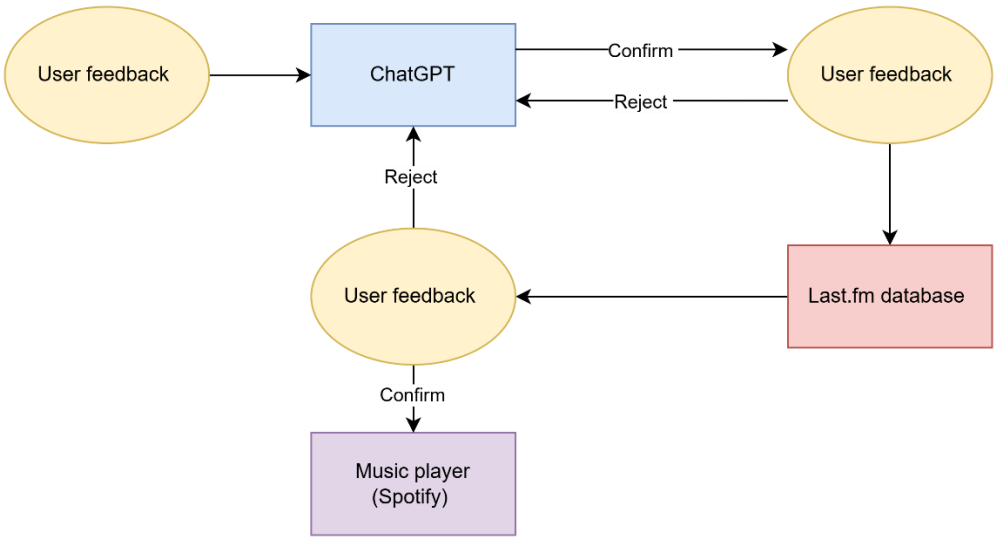

Figure 1-High-Level System Architecture Diagram

3.3: ChatGPT (Chat_Bot) Emotion detection module.

The ChatGPT module is responsible for engaging the user in a short conversation to extract both their **current mood** and **desired emotional state**. Based on this, it generates a list of 10 mood tags sourced from the Last.fm tagging system, each with a corresponding weight. These are stored

in a structured JSON file which becomes the input for the music recommendation module.

This conversational interaction allows the system to respond to ambiguous or layered emotional input and produces a clean, parseable output that links directly to mood-based music retrieval.

A General idea of how this system works can be seen below in figure 2

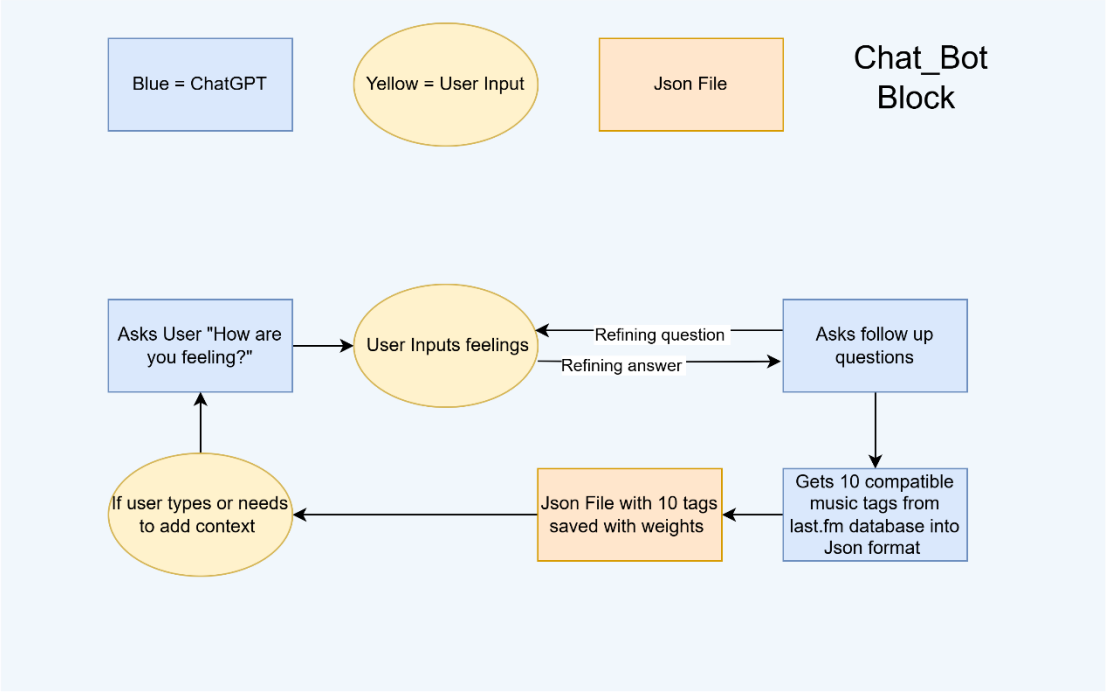

Figure 2-ChatGPT Chat_Bot Block: User emotion input processed into structured JSON tags for music selection.

3.3.1 Prompt Engineering

The project relied heavily on effective prompt design to get reliable, machine-usable JSON from ChatGPT.

In **Semester 1**, the original prompt was designed to map emotional input to **Spotify audio features** the full prompt can be seen in Appendix A. It included instructions for generating two JSON objects (combined features and individual emotion mappings), along with empathetic natural language responses. While useful in theory, this format caused persistent issues:

- Responses were often wrapped in markdown or explanatory text.
- JSON formatting was inconsistent, leading to frequent parsing errors.

- Too many responsibilities were combined in one output.

These issues reflect limitations noted in the Theoretical Background (Section 2.1), where over-complicated prompts reduce output consistency in real-time systems. In Semester 2, a **new prompt** was introduced, specifically

```
You are an AI Assistant that helps users map their emotions to audio features for music selection.

1. Ask the user how they are feeling.

2. Ask follow-up questions until the user confirms a final emotion or set of emotions.

3. Once confirmed, identify:

   - Final emotional state(s)

   - From Last.fm get 10 compatible mood tags matching the emotional need

   - Suggested genres and weighted tags (to prioritise which ones should be queried first)

4. Return ONLY the final emotion(s) and corresponding audio features in valid, parseable JSON format — no explanations, comments, or extra text.

5. Do not wrap the JSON in quotes, markdown, or triple backticks. Just output raw JSON starting with `{`.

6. If the user types after this, repeat steps 1–6 again.

Use this structure as your reference:

{

  "current_mood": ["..."],

  "desired_mood": ["..."],

  "recommended_tags": [

    {"tag": "...", "weight": 0.9},

    {"tag": "...", "weight": 0.7}

  ],

  "suggested_genres": ["..."]

}
```

designed for use with Last.fm. It produces a single, raw JSON block with no extra commentary. The full prompt is shown below:

3.3.2 Code implementation.

This script delivers the core functionality outlined in section 1.4 and 1.5, where real-time emotion detection is described as the basis for adaptive music selection. It also directly implements the logic shown in the **Chat_Bot Block Diagram** (Figure 2), where user input is transformed into machine-usable tag data.

The assistant is configured with a predefined prompt (see Section 3.3.2), designed to return only raw JSON.

The ChatGPT module is built using OpenAI's Assistants API. The following key functions from the OpenAI Python SDK are used to implement the conversation and JSON generation:

- **OpenAI Client Initialization**
  The OpenAI() class is used to authenticate with the API key and link the script to the configured Assistant.
- **Thread Creation**
  A persistent thread is created using client.beta.threads.create().
  This allows the Assistant to handle multi-turn conversations, remembering the user's emotional input across multiple messages.
- **Message Sending and Polling**
  User messages are added to the conversation using messages.create().
  The Assistant processes the message through runs.create_and_poll(), which waits until the assistant's full response is ready.
- **Response Retrieval**
  The latest Assistant message is retrieved with messages.list().
  The textual content is extracted for processing.
- **JSON Parsing and File Saving**
  The Assistant's output is parsed as JSON using json.loads().
  If successful, the resulting structured data is saved to assistant_output.json, which is used by the next stage of the system.

- **Error Handling**
  If the assistant output is not valid JSON, the raw text is returned without saving.
  This prevents system crashes and allows the conversation to continue without interruption.

All core functionality is implemented using standard OpenAI SDK methods and basic Python JSON handling.
Further setup details, such as assistant configuration and authentication, follow OpenAI's public documentation.

3.4: Last.fm (Mood_music) Music recommendation module.

The Mood_Music module is responsible for converting the structured emotional tags generated by ChatGPT into a list of songs using the Last.fm API. It processes the JSON output from the previous module (assistant_output.json), extracts ten weighted mood tags, and uses these to query Last.fm for matching tracks. The resulting song list is stored in a new file, song_list.json, which is then passed to the playback module for queuing.

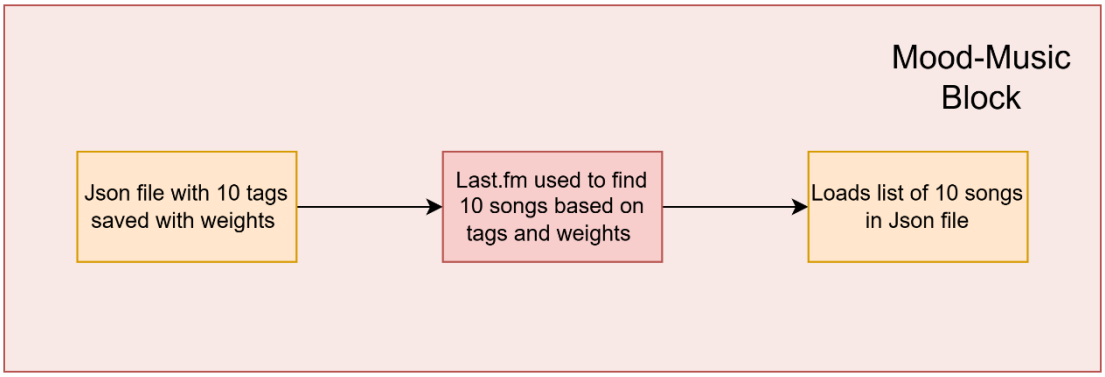

Figure 3-Mood-Music Block: Last.fm receives weighted mood tags and returns a list of 10 recommended songs.

3.4.1: Why Last.fm

Last.fm was selected for its simplicity, open access, and alignment with the system’s modular structure. The tag-based API made it possible to query mood-specific tracks directly using the output from ChatGPT, without requiring additional machine learning or preprocessing.

The platform’s reliance on community-generated tags aligned well with the project's emotional intent, enabling music recommendations based on how listeners perceive songs rather than technical audio features (Like Spotify) Last.fm’s API also integrated smoothly with the system's JSON workflow, supporting lightweight, real-time operation.

A more detailed evaluation of observed limitations and system behaviour is provided in Section 4.

3.4.2 Tag to Track Mapping logic.

This module reads the recommended_tags list from assistant_output.json, which includes ten mood tags with associated weights. These tags are sorted by descending weight and used to query Last.fm using its public tagging endpoints.

Tracks returned by each tag are collected into a list. Duplicate songs and artists are filtered out to ensure variety. The system stops once ten unique tracks are collected. Each item in the final list includes the track name and artist only, and the entire list is saved in song_list.json for use by the next module.

This logic ensures that stronger emotional tags have a higher influence on the result, while also maintaining a cap of ten songs for consistent user experience.

The weight value assigned to each tag ranges from 0.0 to 1.0 and determines query priority. Tags with the highest weight are queried first. If the top tag returns enough unique results, the system may not need to use lower-weighted tags at all.

There is no scoring system applied to the tracks themselves. The weight is used solely to control the order of tag execution. This keeps the logic simple and avoids the need for complex ranking or feature matching.

3.4.3: Code Implementation.

The mood_music.py script reads assistant_output.json, extracts the mood tags and genres, queries the Last.fm API, calculates scores for each track, and writes the results to song_list.json. To do this, the following steps are performed:

• **Read JSON Input**:
Loads the recommended_tags and suggested_genres fields. Tags are sorted by descending weight to prioritise more relevant moods.

• **Query Last.fm API**:
For each tag, the script calls tag.getTopTracks to retrieve a list of relevant tracks. Results are collected across all tags and grouped by (title, artist).

• **Track Scoring System**:
Each track is assigned a cumulative score based on the weights of the tags it matches. If a track appears under multiple tags, its score increases accordingly.

• **Track Tag Matching**:
For each track, the script calls track.gettoptags to retrieve user-generated tags associated with that specific song. This enables further filtering based on emotional relevance.

• **Genre-Weighting Boost**:
If any of the track's tags match a genre from the suggested_genres list, the track receives a +0.5-score boost. This prioritises songs that align more closely with the user's desired style or context.

• **Sorting by Score**:
All tracks are ranked in descending order of total score. This ensures that the most emotionally and contextually relevant tracks are chosen, rather than just the most popular ones.

• **Filter and Limit**:
Duplicate (title, artist) pairs are removed to ensure diversity. The top 10 unique tracks are selected for output.

• **Save Output**:

The result is saved as a dictionary containing a song_list key. Each entry includes a title and artist field, formatted for use by the Spotify playback module.

Standard JSON and HTTP request libraries are used throughout. All API calls and parameters follow Last.fm's official public documentation. [7]

3.5: Spotify (Music_player) Module.

This module handles the final stage of the pipeline: queuing the recommended songs from song_list.json to the user's Spotify account. Using the Spotify Web API and OAuth authentication, it reads the song list, searches for matching track URIs, and adds them to the active Spotify playback queue. This allows for seamless, real-time delivery of mood-based music recommendations directly into the user's listening session.

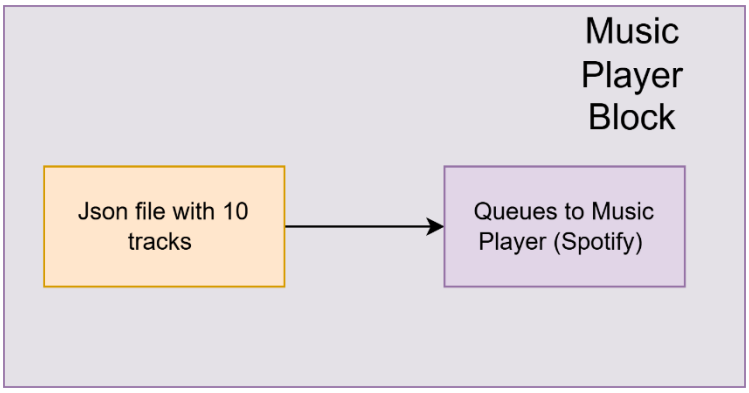

Figure 4-Spotify Block Diagram: Receives song_list.json and queues songs to user's active Spotify session.

3.5.1 Playback Flow and Track Matching Logic

- Reads the song_list.json file produced by the Mood_Music module.
- Uses the Spotify API's search endpoint to find the exact track by title and artist.
- Extracts the track URI from the search result.
- Adds each track URI to the user's active playback queue using the add to queue endpoint.

- Operates sequentially with basic error handling for failed matches or rate limits.

3.5.2 Code Implementation

The playback script uses the spotipy Python library (Spotify API wrapper) for all API operations. Key functional steps include:

• **Authentication**:
OAuth 2.0 is used with the appropriate scopes (e.g. user-modify-playback-state) to allow queueing. The token is refreshed automatically.

• **Read Song List**:
The script loads song_list.json, which contains a list of 10 songs with title and artist.

• **Search for Track URIs**:
For each song, the search() function queries the Spotify API and returns a list of potential matches. The best match (usually the first) is selected based on exact artist/title.

• **Queue to Playback**:
Each track's URI is passed to the add_to_queue() function. This appends the track to the end of the user's current listening queue.

• **Error Handling**:
Tracks that cannot be matched or queued are skipped. Log output is stored for tracking failures.

All Spotify integration follows the official Web API documentation and Spotipy library conventions.

3.7: GUI Module.

The GUI module provides the front-end interface for the system, enabling the user to interact with ChatGPT, trigger music generation, and monitor the system's backend output in real-time. It is built using Python's tkinter and ttk libraries, with additional support from the threading module to maintain responsiveness during API operations. The interface was designed

to provide a clear separation between user interaction and system diagnostics.

The layout is divided into two main panels:

- **Left Panel**: Displays the chat interface, where users input their emotional state and receive responses from the chatbot. It also contains control buttons to send messages and add music once mood tags are ready.
- **Right Panel**: Displays system output, including the prompt being used and contents of key files (assistant_output.json, song_list.json) for debugging and transparency.

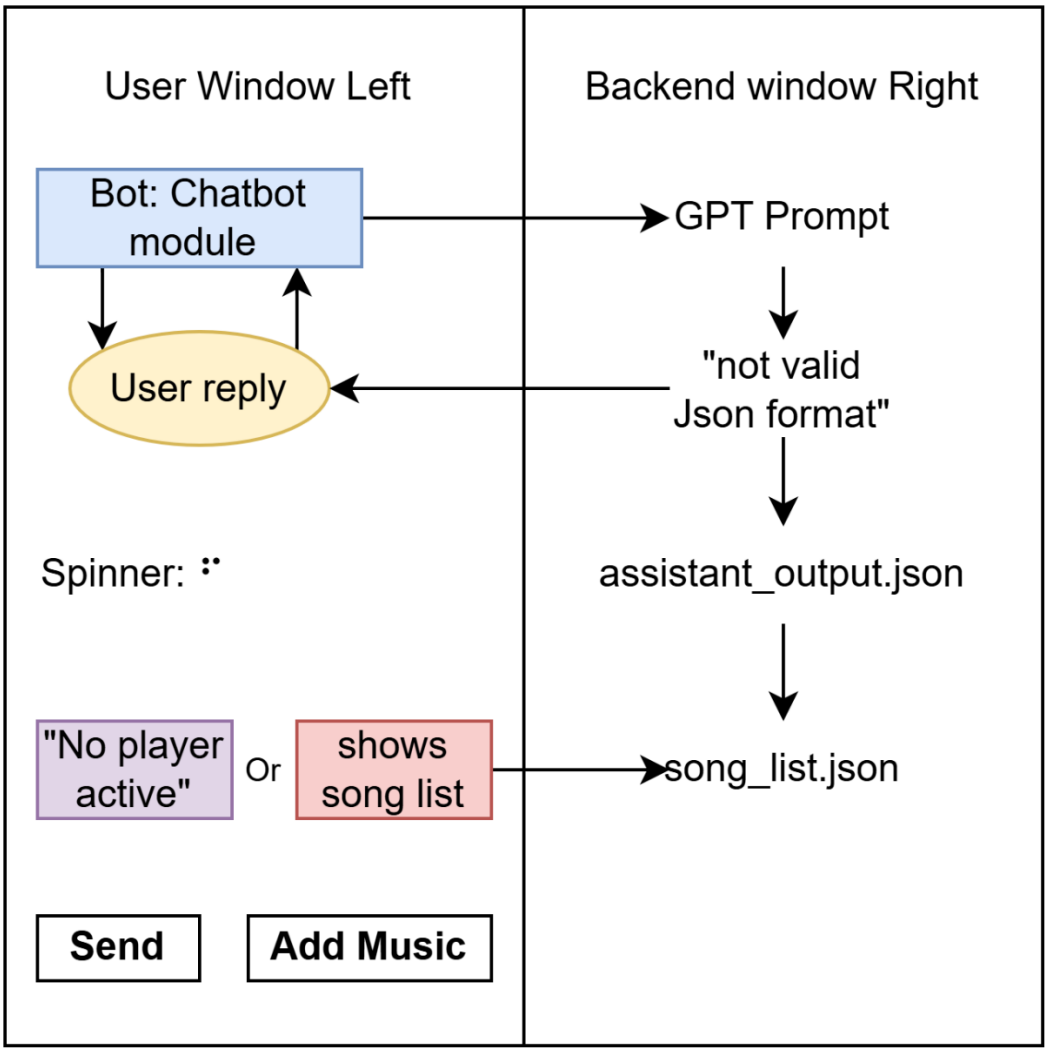

Figure 5-GUI Layout: Left panel for conversation and controls, right panel for logs and outputs.

3.7.1: GUI Interaction flow.

Users begin by typing emotional input into the text entry field. The chatbot responds in a conversational format, using a simulated typing effect to enhance realism. Once ChatGPT outputs a valid structured JSON, the system saves it to assistant_output.json and enables the "Add Music" button. This gives users control over when to trigger the music generation process.

Upon clicking "Add Music", the system generates a list of 10 songs using the Last.fm API and sends them to Spotify for queuing. The contents of both the mood JSON and the song list are displayed in the log panel for verification.

3.7.2: Code Implementation.

The GUI logic is divided into functional areas, each implemented using standard Python libraries. Responsiveness is maintained using threading, while tkinter handles the layout and interface components.

- **Left Panel – Chat Interaction**

Uses tk.Text, tk.Entry, and tk.Button widgets for input and output. Tags are applied using .tag_config() to style user and bot messages. This setup allows real-time feedback and styled chat content [8].

- **Typewriter Effect and Message Queue**

Bot messages are added to a queue and displayed one character at a time using the .after() method. This creates a simulated typing effect without blocking the main GUI thread [8].

- **Spinner Animation**

A separate thread cycles through Unicode spinner characters to show progress while awaiting API responses. The spinner position is tracked and updated in the chat window in real time [8].

- **Right Panel – Logs and JSON Viewer**

Displays static and dynamic content using another tk.Text widget. JSON output is loaded using json.load() and formatted with json.dumps(indent=2) for clarity [9].

• **Threading for Responsiveness**

Time-consuming operations such as chatbot response, song generation, and Spotify queuing run in daemon threads using Python's threading.Thread() [10]. This prevents the UI from freezing and allows asynchronous operations.

• **State Management and Controls**

The "Add Music" button is managed using .config(state=tk.DISABLED) and is only enabled after valid JSON is returned. All error states are handled gracefully and logged to the right panel.

4. Implementation/Simulation/Test/ Realisation

4.1: Full-System Testing.

Full-system testing was carried out using the GUI as the main control point. Each test began with user input and ended with a track queue update on Spotify. The goal was to confirm that all modules worked together as intended.

Three types of tests were used:

- **Functional Tests:** Checked that emotional input triggered valid JSON generation, mood-based song selection, and Spotify queueing. These were used to verify the standard flow under normal conditions.
- **Edge Case Tests:** Included short, unclear, or mixed emotional inputs. These were used to test how well the system handled vague or complex user messages.
- **Failure Tests:** Simulated API failures, empty JSON responses, and missing data. These helped verify that the system could handle errors without crashing.

Each test was run end-to-end. Output files and the GUI log panel were used to check system behaviour at each stage. Testing confirmed whether the

correct files were created, if the song list matched expected tags, and whether Spotify responded correctly.

4.2: Implementation Revisions.

During testing, several issues were identified and addressed to improve system reliability and responsiveness.

- **Threading Conflicts:** The GUI would freeze while waiting for API responses. This was fixed by running long processes (e.g. ChatGPT, Last.fm, Spotify calls) in daemon threads. This allowed the interface to remain responsive during background tasks.
- **JSON Timing Delay:** In some cases, the system tried to read the JSON file before it was fully written. A delay was added to check that the file existed and contained valid data before continuing.
- **Spotify Lookup Failures:** Some songs could not be found using the default search format. The search function was updated to handle partial matches and skip problematic entries without crashing the queue.
- **Redundant Tags:** Some emotional tags returned little or no results. The system was updated to sort tags by weight and fall back to lower-weighted tags when needed, ensuring at least 10 songs could be returned.

These changes were made based on observed failures during integration testing and improved the overall stability of the system.

4.3: User flow and Behavioural Testing.

Behavioural testing focused on verifying that the GUI correctly triggered the system pipeline, managed user interaction, and displayed accurate system states. The following areas were tested:

- **Chat Response Timing:** Chatbot messages were delivered using a typewriter effect and spinner animation. Tests confirmed that responses were readable and non-blocking. Spinner animation ran on

a separate thread and was correctly stopped once a valid JSON response was received.

- **Button State Management:** The “Add Music” button was disabled by default and only enabled after a valid assistant_output.json file was saved. Tests confirmed this prevented premature activation of the music generation pipeline.
- **Error Handling in GUI:** If ChatGPT returned invalid or non-JSON output, the system displayed a warning in the log panel and allowed the user to continue chatting. This was tested using malformed prompts, and the system remained stable.
- **Music Generation and Playback:** When “Add Music” was clicked, the system ran the Last.fm query and queued songs to Spotify in the background. The GUI showed the contents of song_list.json and displayed a final song list in the chat window. Tests confirmed that output formatting, song ordering, and Spotify queueing all worked as expected when valid inputs were provided.
- **Repeated Use:** The interface was tested with multiple consecutive emotion inputs. The system successfully generated new recommendations each time without requiring a restart. However, repeated use with the same emotional input sometimes returned overlapping tracks. This was expected due to static tag-based queries and was not treated as a bug.

4.4 Execution Logging and Validation Strategy.

System execution was validated using real-time logs displayed on the GUI’s right-hand panel. This panel provided continuous feedback across all key stages of the pipeline—detecting when output files were generated, when errors occurred, and when final playback was confirmed.

During testing, the GUI was used exclusively to monitor backend performance:

- When the chatbot was active, the system checked each response for valid JSON.

- If a response was incomplete or mid-conversation, the right panel displayed:

```
Chatbot: This is not a JSON format, Waiting for JSON Format...:
Chatbot: This is not a JSON format, Waiting for JSON Format...:
```

Once ChatGPT returned a valid JSON structure, it was saved to assistant_output.json and immediately shown in the GUI's output box.

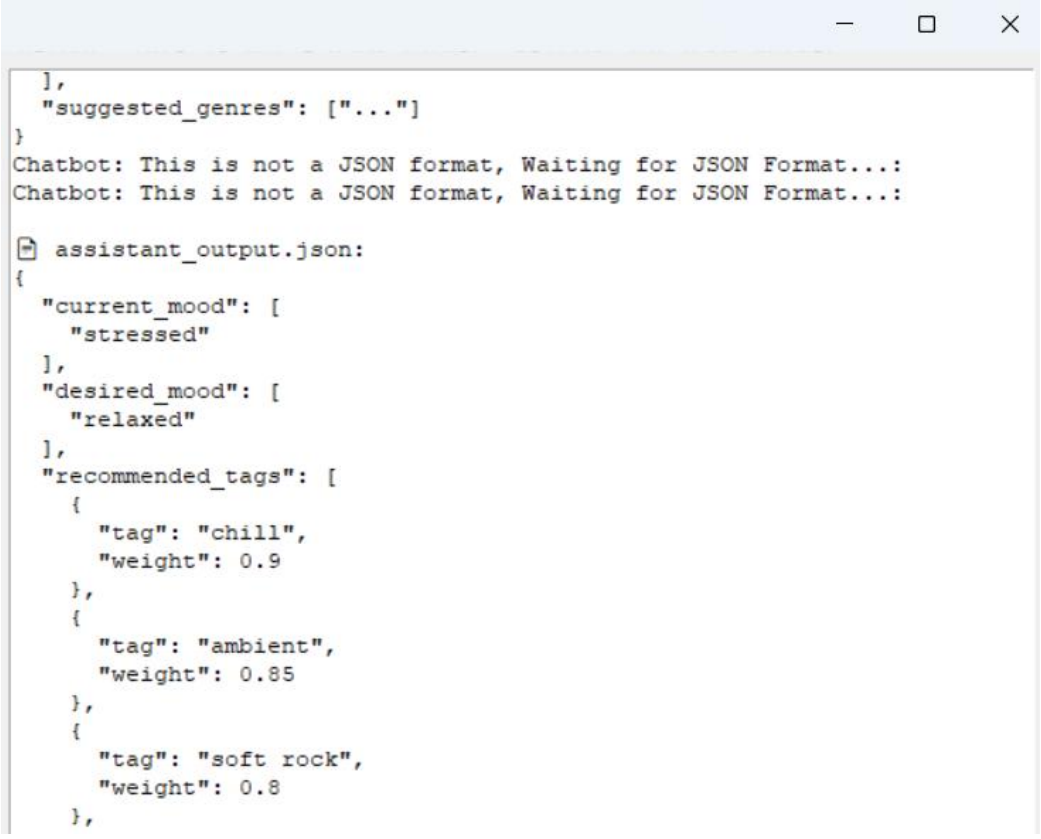

Figure 6-GUI Log Panel Showing JSON Detection and File Confirmation

This screenshot shows how the system acknowledged valid emotional input and confirmed that `assistant_output.json` was ready.

After confirmation, the user was prompted to click "Add Music". The logging system then displayed one of two outcomes:

- If no active Spotify device was detected, both left and right panels showed an error message indicating the issue.

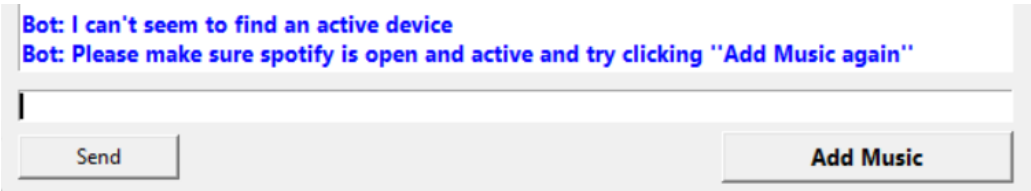

Figure 7-Error message shown to User (left side)

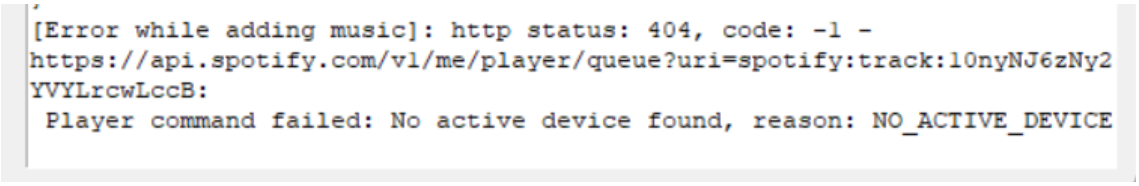

Figure 8-Error message shown for debugging (on right side)

- If playback was active, the system ran the recommendation pipeline. The resulting song_list.json file was printed in the log, followed by a list of the 10 queued songs with artist and title.

```
  ]
}

song_list.json:
{
  "song_list": [
    {
      "title": "Sparks",
      "artist": "Coldplay"
    },
    {
      "title": "I'm Yours",
      "artist": "Jason Mraz"
    },
    {
      "title": "Lush Life",
      "artist": "Zara Larsson"
    },
    {
      "title": "How to Save a Life",
      "artist": "The Fray"
    },
    {
      "title": "Sweater Weather",
      "artist": "The Neighbourhood"
    },
    {
      "title": "Island in the Sun",
      "artist": "Weezer"
    },
    {
      "title": "Karma",
      "artist": "Taylor Swift"
    },
    {
```


Figure 9-song_list.json Displayed After Track Generation

This image shows the generated song list printed in the GUI output panel, confirming the handoff from Last.fm to Spotify.

```
Added to queue: Sparks by Coldplay
Added to queue: I'm Yours by Jason Mraz
Added to queue: Lush Life by Zara Larsson
Added to queue: How to Save a Life by The Fray
Added to queue: Sweater Weather by The Neighbourhood
Added to queue: Island in the Sun by Weezer
Added to queue: Karma by Taylor Swift
Added to queue: Teardrop by Massive Attack
Added to queue: Everything in Its Right Place by Radiohead
Added to queue: Nude by Radiohead
🚀 Spotify queue complete.
```

Figure 10-Confirmation of Track Queueing to Spotify

Final log entries confirm that each recommended track was queued, showing both system status and playback completion messages.

This logging structure was essential for tracking progress and debugging failures. It allowed for real-time validation of file outputs, API calls, and end-to-end system flow, all from within the GUI.

5. Results

5.1 Emotion Detection Output

The system was tested using a range of emotional inputs to evaluate its ability to extract mood states, assign weighted tags, and suggest appropriate genres. The following example shows a complete conversation flow from input to final song list generation.

Chatbot:
"Hello! How are you feeling today?"

User:
"I'm feeling really drained but also kind of hopeful."

Chatbot:
"Thanks for sharing. Would you say you're mostly feeling mentally tired, emotionally low, or just in need of a reset?"

User:
"Yeah, mentally tired mostly, but I do want to feel better — like I'm trying to stay positive."

Chatbot:
"Got it. So, you're feeling a bit burnt out, but also hopeful and looking for something uplifting. Does that sound about right?"

User:
"Exactly. Tired but optimistic."

Table 1-Sudo Interaction between Chatbot and User

From this interaction, the system generated the following structured JSON output:

```
{
  "current_mood": ["tired", "hopeful"],
  "desired_mood": ["uplifted", "motivated"],
  "recommended_tags": [
    {"tag": "calm", "weight": 0.9},
    {"tag": "soothing", "weight": 0.85},
    {"tag": "hopeful", "weight": 0.8},
    {"tag": "uplifting", "weight": 0.75},
    {"tag": "motivational", "weight": 0.7}
  ],
  "suggested_genres": ["ambient", "instrumental", "soft pop"]
}
```

Figure 11-assistant_output.json for 'Tired but Hopeful' Input

This output was passed to the mood_music module, which queried Last.fm using the top-weighted tags. The following song list was returned:

```
{
  "song_list": [
    { "title": "Weightless", "artist": "Marconi Union" },
    { "title": "Rise Up", "artist": "Andra Day" },
    { "title": "Let It Be", "artist": "The Beatles" },
    { "title": "Lovely Day", "artist": "Bill Withers" },
    { "title": "Don't Stop Believin'", "artist": "Journey" }
  ]
}
```

Figure 12-song_list.json returned from Last.fm using mood tags

Note: for both figures 11 and 12 the length of tags/songs was cut down from 10 to 5 for easier use and to fit in the report easier.

5.2 Testing Multi-Emotion Inputs Against Prompt Logic

The assistant was tested with increasingly complex emotional expressions, including inputs that mention multiple or conflicting emotions. The table below tracks how well the assistant followed each step of the defined prompt:

User Input	Step 1: Asked how user feels	Step 2: Asked follow-ups & confirmed	Step 3: Identified emotions, tags, genres	Step 4: Returned valid raw JSON	Prompt Completed?	Notes
I'm happy.	Yes	Yes	Yes	Yes	Yes	Handled cleanly; prompt fully executed
I'm tired but also hopeful.	Yes	Yes	Yes	Yes	Yes	Dual-emotion inputs processed correctly
I'm frustrated, anxious, and kind of sad but trying to be optimistic.	Yes	Yes	Yes	Yes	Yes	Complex emotion handled; tag weighting slightly skewed to negative
I feel lost, stuck, and drained but also kind of excited for change.	Yes	Yes	Yes	Yes	Yes	Long emotional input processed; final JSON was usable
Meh.	Yes	Yes	Yes (low-quality tags)	Yes	Yes	Prompt completed but tags were generic (e.g. *neutral*, *chill*)
I don't really know.	Yes	Looped repeatedly	No	No	No	Bot kept asking clarification questions without detecting emotion
I had toast and coffee earlier.	Yes	Looped repeatedly	No	No	No	Bot did not detect emotion; stayed in step 1 loop

Table 2-Prompt Step Completion vs Input Complexity

This table shows that the assistant consistently followed all steps when emotional content was present, even if multiple emotions were included. For valid inputs, the final JSON was always returned in the correct format as specified.

However, in cases with **non-emotional or vague responses**, the assistant failed to progress past step 2. It looped on clarifying questions without ever generating an output, showing that while the prompt prevents invalid data, it also lacks a mechanism to time out or reframe when a user stays uncooperative or ambiguous.

5.3: Error Handling Outputs

The system was tested under several common failure conditions to ensure it could handle invalid data, missing files, or broken API responses without crashing. In all cases, error handling worked as intended, and the GUI remained stable.

If ChatGPT failed to return valid JSON, the system logged the issue and allowed the user to continue the conversation. When Spotify was inactive or a track could not be found, appropriate messages were shown in the chat or log panels, and the rest of the process continued.

The table below summarises the scenarios tested and the system's behaviour:

Test Scenario	**Trigger Condition**	**System Response**	**Outcome**
Invalid JSON from ChatGPT	ChatGPT returns unstructured or extra text	User prompted to continue chatting	JSON generated on retry
Add Music before chatting	User tries to click "add music" before music Json creation.	Button only allows users to enter after assistant_output has been filled	Prevented invalid pipeline
Spotify not active	Spotify not open or no playback device	Chatbot displays guidance message	No crash; user informed
Song not found on Spotify	Track not found using title/artist match	Track skipped; others added	Queue completes as expected
File read error (e.g. corrupted JSON file)	File missing or unreadable during display	Error shown in right log panel	GUI remains stable

Table 3-Testing Summary

6. Analysis.

This section evaluates how well the system achieved its intended objectives, based on the implementation results. The key features tested were emotion recognition, tag mapping, song recommendation, and system stability. Overall, the system performed reliably in most real-world use cases, with limitations observed only under edge conditions.

6.1 Conversational Emotion Detection

ChatGPT was able to extract emotional intent from user input using structured prompt logic. For clear and moderately complex emotional inputs, the assistant asked appropriate follow-up questions and returned clean JSON formatted output. In cases where multiple or conflicting emotions were present (e.g. *anxious but hopeful*), the system still followed the prompt and produced usable output.

However, for vague or non-emotional inputs, the assistant entered clarification loops and failed to return structured data. This behaviour was consistent with the prompt rules, which prevented invalid JSON but offered no fallback when user input lacked emotion. While this did not crash the system, it halted progress until the user rephrased.

6.2 Tag-Based Song Recommendation

Once valid JSON was detected, tag-based music generation worked consistently. Last.fm was able to map emotional tags to relevant songs, and the weighting system prioritised high-intensity emotions. Songs returned were diverse, and genres generally aligned with user intent.

However, some emotional weightings skewed heavily toward one side—especially when negative emotions like *anxious* or *drained* were present. This occasionally diluted the effect of the desired mood (e.g. *hopeful*, *uplifting*). While the system returned 10 unique songs as expected, emotional balance was not always ideal. This suggests a possible improvement area in the weighting logic or tag filtering.

6.3 System Stability and Error Management

Error handling was tested under various failure scenarios: invalid ChatGPT output, missing files, inactive Spotify sessions, and unfound tracks. In all cases, the GUI remained stable. Messages were shown clearly to the user, and logs provided sufficient debugging information. The use of threading ensured the chat interface stayed responsive, even during long API calls or file access.

The Add Music button was correctly disabled until JSON generation was complete, and runtime errors (e.g. Spotify inactive) were caught and

displayed in both chat and log panels. No full system crashes occurred during any tests.

6.4 Prompt Effectiveness and Limits

The refined prompt used in semester two successfully enforced a clean JSON format without markdown or commentary. The structure matched the required format for downstream processing. The assistant strictly adhered to the flow (ask how you're feeling → clarify → generate JSON), which helped reduce inconsistent behaviour.

However, the same strictness became a weakness when inputs were emotionally vague or off-topic. The assistant would repeat clarification questions indefinitely, never moving to JSON generation. This revealed the need for a timeout, re-prompt, or broader fallback logic to move conversations forward.

6.5 Summary Table of Observed Performance

Component	What Worked Well	Limitations Noted
ChatGPT Prompt	Accurate detection, clean JSON, adaptive follow-ups	Loops indefinitely on vague inputs
Mood Tag Weighting	Generated relevant song lists aligned to emotional state	Strong negative inputs occasionally overpower desired mood
Last.fm Integration	Returned full track lists with varied artists and genres	Tag results sometimes repeated or generic on low-weight tags
Spotify Queueing	Playback worked if Spotify was active, songs queued successfully	No device feedback; tracks not found were silently skipped
GUI Responsiveness	Spinner, typewriter effect, and real-time logging performed smoothly	No real-time Spotify queue status shown; manual verification needed
Error Handling	Graceful on JSON errors, missing files, API failures	No retry logic or advanced fallback for Spotify or Last.fm failures

Table 4-Analysis Summary

7. Conclusion

sThe aim of this project was to develop a functional prototype of an emotion-based music recommendation system using ChatGPT for conversational mood detection and Last.fm for music tagging. This was achieved through a modular pipeline that successfully converted user emotional input into mood-based playlists, with real-time playback delivered through Spotify.

Each objective outlined in the introduction was addressed:

- A working chatbot was implemented using OpenAI's Assistants API, capable of detecting user emotions through natural conversation.
- Emotional states were mapped to weighted Last.fm mood tags and used to query music recommendations aligned with the user's mood.
- The system was built using a modular structure, allowing the ChatGPT, Last.fm, and Spotify components to function independently and communicate through structured JSON.
- A fully functional GUI was created to handle all user input, system output, and interaction flow.

Testing confirmed that the system worked reliably for most user inputs. Clean JSON was generated in line with the prompt structure, song lists were returned with good diversity, and error handling was effective in preventing crashes or system failure.

However, the project also identified some clear limitations. The assistant struggled with vague or non-emotional input and lacked a mechanism to resolve stuck conversations. In some cases, emotionally conflicting inputs led to skewed tag weighting and mixed music recommendations. There was no memory between sessions or user-specific learning.

Despite these limitations, the prototype is considered successful. It demonstrated that a conversational, real-time emotion-to-music system is technically feasible using existing tools and APIs. The modular architecture allows for easy improvement and scaling in future development.

References

[1] A. C. R. S. N. K. Gupta, “ChatGPT: Exploring the Capabilities and Limitations of a Large Language Model for Conversational AI,” 2023 International Conference on Advances in Computation, Communication and Information Technology (ICAICCIT), 2023.

[2] F. A. Z. T. L. A. M. M. A. P. a. Z. Z. O. Mubin, “Tracking ChatGPT Research: Insights From the Literature and the Web,” IEEE Access, vol. 12, p. 30532, 2024.

[3] Z.-C. C. C.-H. H. Y.-C. Chen, “Music Mood Classification System for Streaming Platform Analysis via Deep Learning Based Feature Extraction,” IEEE International Conference on Consumer Electronics-Taiwan (ICCE-TW), Taipei, Taiwan.

[4] E. Çano and M. Morisio, “Music Mood Dataset Creation Based on Last FM Tags,” Proceedings of the 2017 International Conference on Computer Science & Information Technology, pp. 15-26, 2017.

[5] Spotify, “Changes to the Web API,” [Online]. Available: https://developer.spotify.com/blog/2024-11-27-changes-to-the-web-api.

[6] Apple Developer Documentation, "App Store Connect API," [Online]. Available: https://developer.apple.com/documentation/appstoreconnectapi.

[7] Last.fm Developer Documentation, "Last.fm API," [Online]. Available: https://www.last.fm/api.

[8] Python Software Foundation, "tkinter — Python interface to Tcl/Tk," [Online]. Available: https://docs.python.org/3/library/tkinter.html

[9] Python Software Foundation, "json — JSON encoder and decoder," [Online]. Available: https://docs.python.org/3/library/json.html

[10] Python Software Foundation, "threading — Thread-based parallelism," [Online]. Available: https://docs.python.org/3/library/threading.html